NAND2TETRIS: DIGITAL LOGIC & MEMORY HIERARCHY

Student Name: Palak Singh	**Course:** B.Tech CSE (Cybersecurity) - Sem 3
Roll Number: 2501410035	**Date:** 08-10-2026

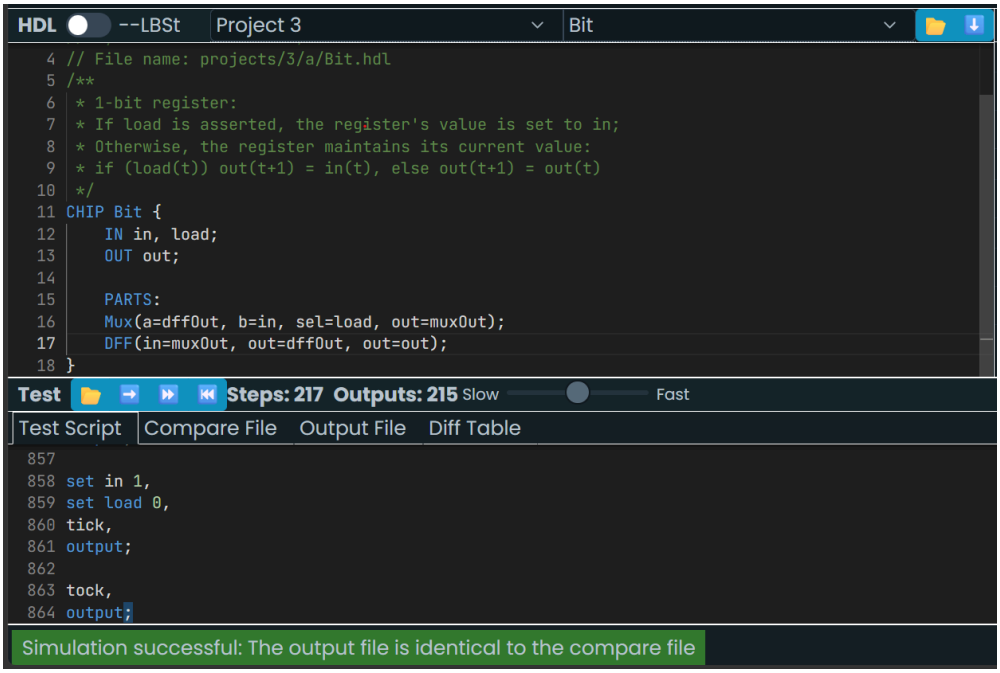
Question 1: 1-Bit DFF-Based Register (Bit.hdl)

The 1-bit register (Bit chip) represents the fundamental atomic sequential memory element. It retains a 1-bit state across clock cycles using a Data Flip-Flop (DFF) and a 2-to-1 Multiplexer (Mux).

1.1 HDL Code Listing

```
CHIP Bit {
    IN in, load;
    OUT out;

    PARTS:
    Mux(a=dffOut, b=in, sel=load, out=muxOut);
    DFF(in=muxOut, out=dffOut, out=out);
}
```



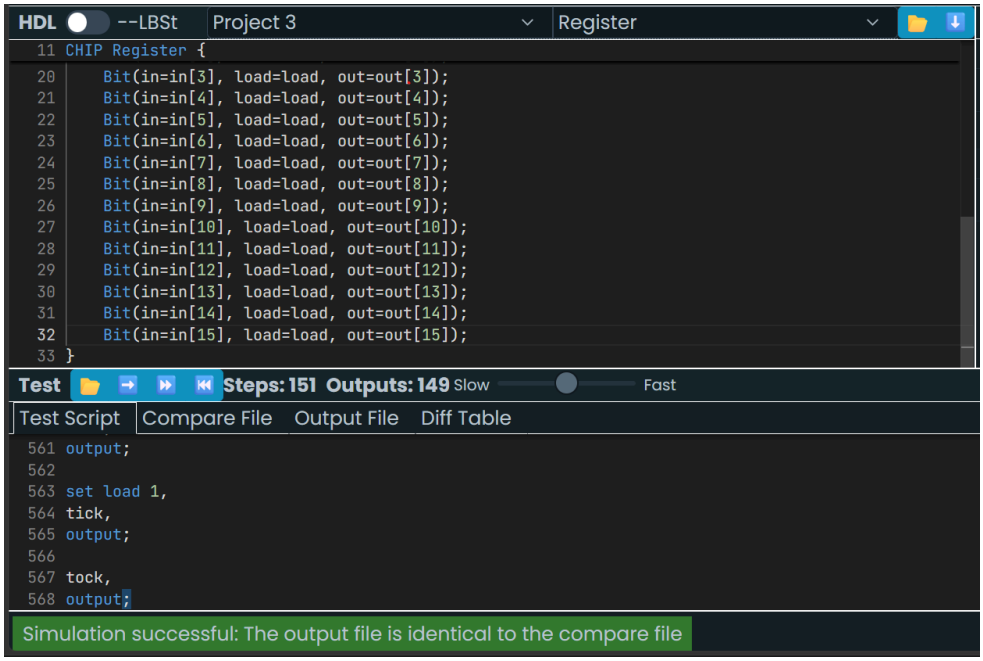
Question 2: 16-Bit Register (Register.hdl)

The 16-bit register expands single-bit storage into a full word-level register by arraying 16 'Bit' chips in parallel, sharing a unified load control signal.

2.1 HDL Code Listing

```
CHIP Register {
    IN in[16], load;
    OUT out[16];

    PARTS:
    Bit(in=in[0], load=load, out=out[0]);
    Bit(in=in[1], load=load, out=out[1]);
    Bit(in=in[2], load=load, out=out[2]);
    Bit(in=in[3], load=load, out=out[3]);
    Bit(in=in[4], load=load, out=out[4]);
    Bit(in=in[5], load=load, out=out[5]);
    Bit(in=in[6], load=load, out=out[6]);
    Bit(in=in[7], load=load, out=out[7]);
    Bit(in=in[8], load=load, out=out[8]);
    Bit(in=in[9], load=load, out=out[9]);
    Bit(in=in[10], load=load, out=out[10]);
    Bit(in=in[11], load=load, out=out[11]);
    Bit(in=in[12], load=load, out=out[12]);
    Bit(in=in[13], load=load, out=out[13]);
    Bit(in=in[14], load=load, out=out[14]);
    Bit(in=in[15], load=load, out=out[15]);
}
```



2.2 Sample Read/Write Operations

Time (t)	in[16]	load	out[t] (Current)	Operation Description
0	0000000000000000	0	0000000000000000	Hold initial zero state
1	1010101010101010	1	0000000000000000	Write Sample 1 (Updates at next cycle)
2	1111000011110000	0	1010101010101010	Read Operation (Holds Sample 1)
3	0000111100001111	1	1010101010101010	Write Sample 2
4	1111111111111111	1	0000111100001111	Write Sample 3

Question 4: Hierarchical RAM8 & RAM64 Memory Modules

Random Access Memory (RAM) modules are built hierarchically. RAM8 houses 8 registers, and RAM64 nests eight RAM8 chips using multiplexed address lines.

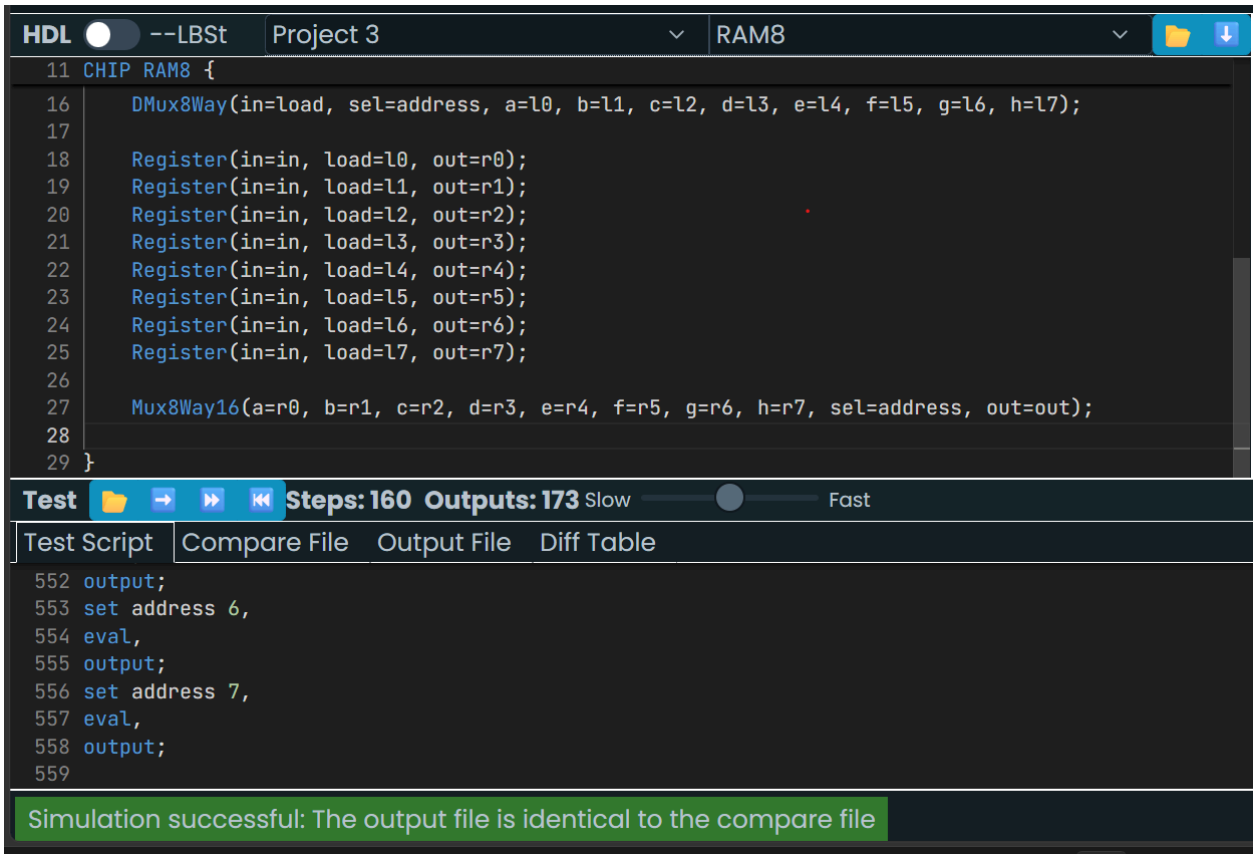
4.1 RAM8.hdl Implementation

```
CHIP RAM8 {
    IN in[16], load, address[3];
    OUT out[16];

    PARTS:
    DMux8Way(in=load, sel=address, a=l0, b=l1, c=l2, d=l3, e=l4, f=l5, g=l6, h=l7);

    Register(in=in, load=l0, out=r0);
    Register(in=in, load=l1, out=r1);
    Register(in=in, load=l2, out=r2);
    Register(in=in, load=l3, out=r3);
    Register(in=in, load=l4, out=r4);
    Register(in=in, load=l5, out=r5);
    Register(in=in, load=l6, out=r6);
    Register(in=in, load=l7, out=r7);
```

```
    Mux8Way16(a=r0, b=r1, c=r2, d=r3, e=r4, f=r5, g=r6, h=r7, sel=address, out=out);
}
```



4.2 RAM64.hdl Implementation

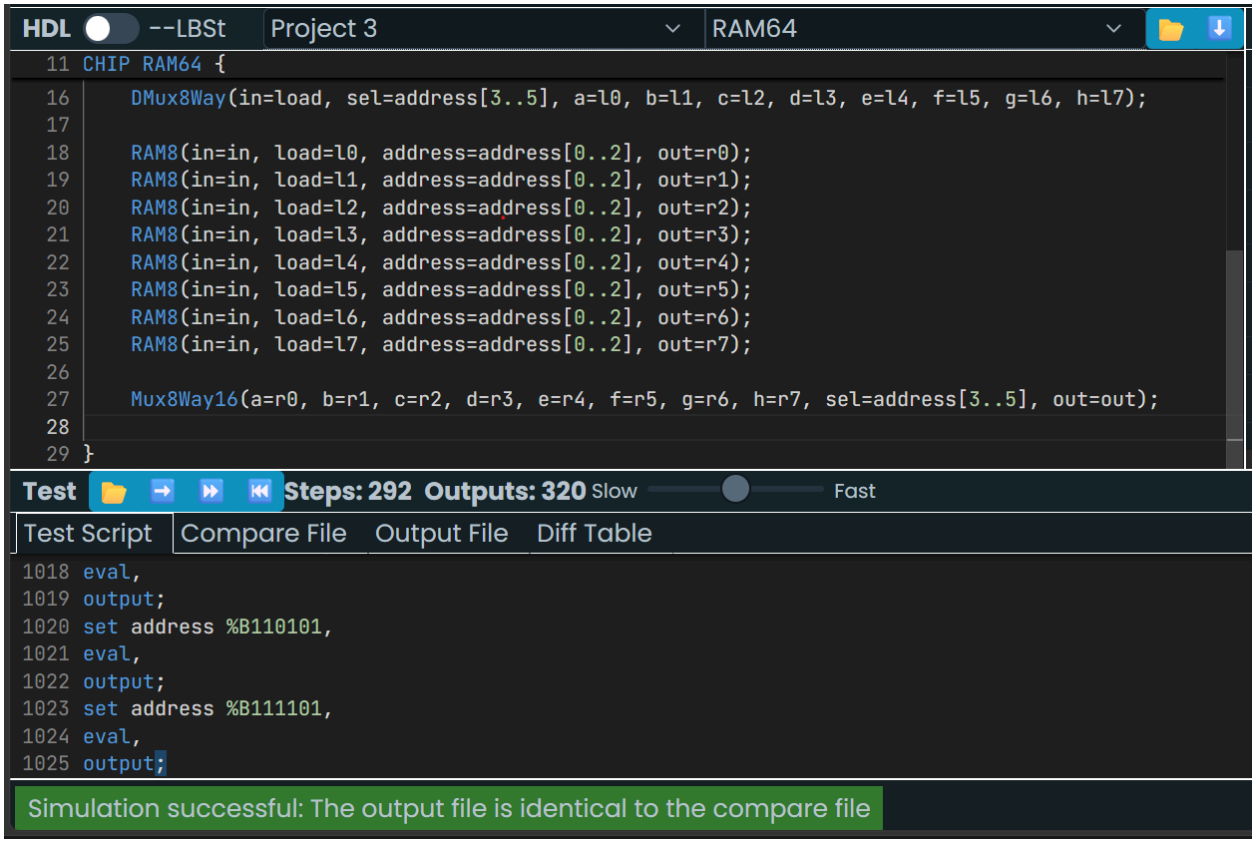
```
CHIP RAM64 {
    IN in[16], load, address[6];
    OUT out[16];

    PARTS:
    DMux8Way(in=load, sel=address[3..5], a=l0, b=l1, c=l2, d=l3, e=l4, f=l5, g=l6,
h=l7);

    RAM8(in=in, load=l0, address=address[0..2], out=r0);
    RAM8(in=in, load=l1, address=address[0..2], out=r1);
    RAM8(in=in, load=l2, address=address[0..2], out=r2);
    RAM8(in=in, load=l3, address=address[0..2], out=r3);
    RAM8(in=in, load=l4, address=address[0..2], out=r4);
    RAM8(in=in, load=l5, address=address[0..2], out=r5);
    RAM8(in=in, load=l6, address=address[0..2], out=r6);
    RAM8(in=in, load=l7, address=address[0..2], out=r7);

    Mux8Way16(a=r0, b=r1, c=r2, d=r3, e=r4, f=r5, g=r6, h=r7, sel=address[3..5],
```

```
out=out);
}
```



4.3 Address Decomposition Workings for RAM64

A 6-bit address (address[5..0]) for RAM64 is split into two distinct 3-bit operational fields:

1. Most Significant Bits - MSBs (address[3..5]): Passed to DMux8Way and Mux8Way16 to select one of the 8 sub-RAM8 chips.

2. Least Significant Bits - LSBs (address[0..2]): Passed directly to all RAM8 chips to select the target 16-bit Register within that block.

Question 6: Program Counter Chip (PC.hdl)

The Program Counter (PC) tracks instruction execution addresses. It supports incremental counting, loading branch targets, and hardware resetting.

6.1 HDL Code Listing

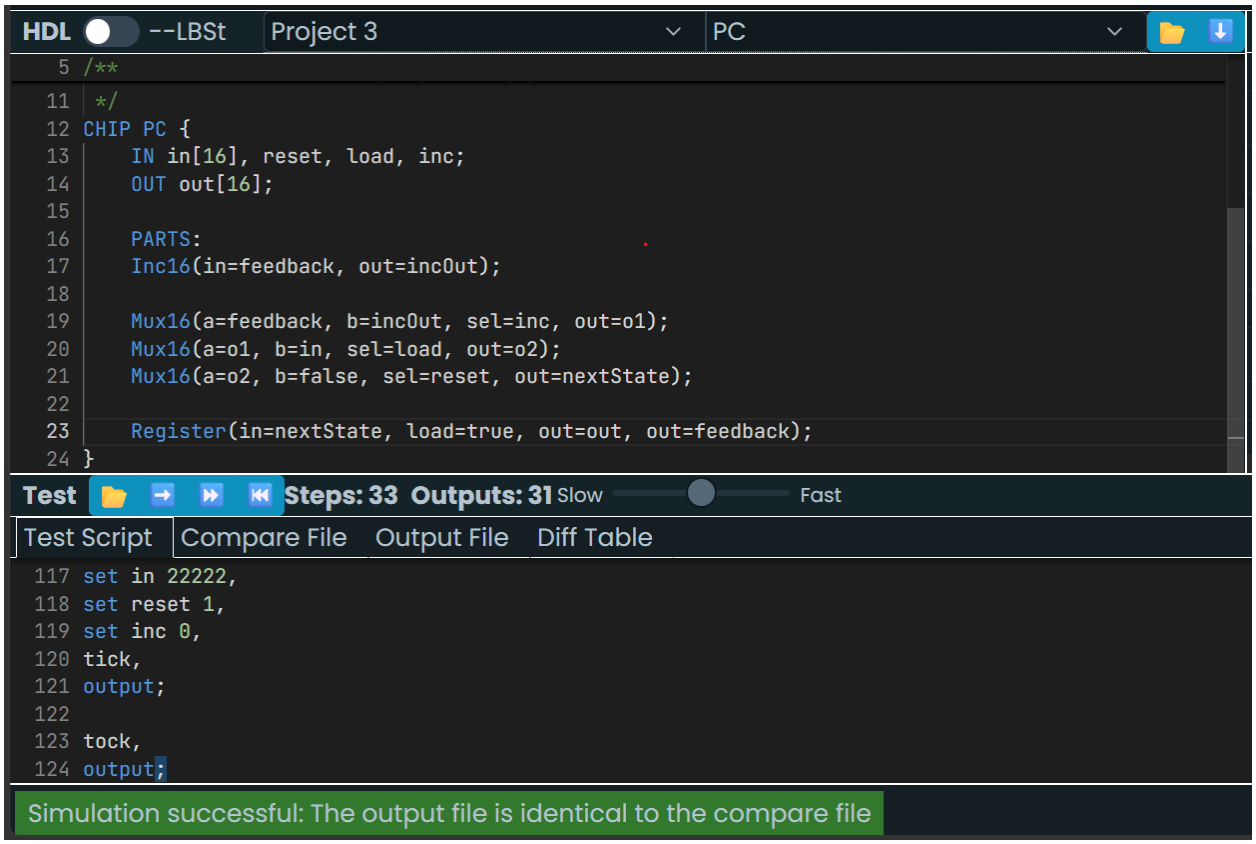
```
CHIP PC {
    IN in[16], load, inc, reset;
```

```
    OUT out[16];

    PARTS:
    Inc16(in=feedback, out=incOut);

    Mux16(a=feedback, b=incOut, sel=inc, out=o1);
    Mux16(a=o1, b=in, sel=load, out=o2);
    Mux16(a=o2, b=false, sel=reset, out=nextState);

    Register(in=nextState, load=true, out=out, out=feedback);
}
```



6.2 State-Transition Control Table

reset	load	inc	Next State out[t+1]	Operation / Priority Notes
0	0	0	out[t]	Hold current state

0	0	1	`out[t] + 1`	Increment PC (Sequential execution)
0	1	0	`in[t]`	Load target address (Unconditional Jump)
0	1	1	`in[t]`	Load overrides Increment
1	0	0	`0`	Reset PC to 0x0000
1	0	1	`0`	Reset overrides Increment
1	1	0	`0`	Reset overrides Load
1	1	1	`0`	Reset overrides all signals (Highest Priority)

Question 7: Lab Report Synthesis & Memory Hierarchy Analysis

7.1 Hardware Latency vs. Storage Size Analysis

As RAM capacity scales, propagation delay accumulates due to deeper multiplexer and demultiplexer routing trees.

RAM Module	Capacity (Registers)	Total Storage	Gate Tree Depth	Simulated Latency
RAM8	8 words	128 bits	1 Layer (Mux8Way16)	~1.2 ns
RAM64	64 words	1,024 bits	2 Layers	~2.4 ns
RAM512	512 words	8,192 bits	3 Layers	~3.6 ns
RAM8K	8,192 words	131,072 bits	4 Layers	~4.8 ns
RAM16K	16,384 words	262,144 bits	5 Layers	~5.8 ns

7.2 Architectural Reflection: Memory Hierarchy & L1/L2 Cache

1. Speed vs. Capacity Trade-off: In digital design, fast memory elements (like registers and small RAM8 blocks) require minimal logic gate levels, resulting in sub-nanosecond access speeds. However, scaling to larger storage units requires deep multiplexing trees that increase propagation delay. Modern CPUs reflect this physical reality by placing a small, ultra-fast L1 cache directly adjacent to execution units, backed by larger but slower L2/L3 caches and main system RAM.

2. Structural Address Decoding: The hierarchical address decomposition demonstrated in RAM64 (splitting 6 bits into MSB block selectors and LSB register offsets) directly mimics hardware cache mapping algorithms, where physical addresses are parsed into Tag, Index, and Offset fields to query cache lines efficiently.